

# CO5\_ASSESSMENT 3: MOCK INTERVIEW SOLUTIONS

Comprehensive Answers for Computer Vision & OpenCV Technical Evaluation

Course: Computer Vision (CO5)

Subject: OpenCV Technical Interview Prep

Total Questions: 25

## QUESTION 1

### What is OpenCV, and why is it commonly used in computer vision applications?

**OpenCV (Open Source Computer Vision Library)** is an open-source real-time computer vision and machine learning software library. It provides over 2,500 optimized algorithms for image processing, video analysis, feature extraction, object detection, and 3D reconstruction.

**Why it is commonly used:**

- **Cross-Platform & Multi-Language Support:** Written in highly optimized C/C++, with wrappers for Python, Java, and MATLAB, running seamlessly across Linux, Windows, macOS, Android, and iOS.
- **Performance & Optimization:** Leverages MMX/SSE instructions, CUDA/OpenCL GPU acceleration, and IPP (Intel Integrated Performance Primitives) for real-time computational throughput.
- **Extensive Ecosystem:** Offers end-to-end functionality from basic I/O and matrix operations to deep learning inference engine (`cv2.dnn`).

## QUESTION 2

### If you are given a folder containing hundreds of images, how would you design an OpenCV pipeline to read and process them efficiently?

To process hundreds or thousands of images efficiently, a robust asynchronous data pipeline is required:

- **Directory Scanning:** Use `pathlib.Path` or `os.scandir()` rather than `os.listdir()` for memory-efficient lazy evaluation.
- **Multithreading / Multiprocessing:** Use Python's `concurrent.futures.ProcessPoolExecutor` or `multiprocessing.Pool` to distribute CPU-bound image transformations across multiple CPU cores.
- **Batch Pre-allocation & Resizing:** Read and immediately downsample images to standard input dimensions (e.g., 640x640) early in memory to reduce downstream memory bandwidth.
- **I/O Optimization:** If reading from disk becomes a bottleneck, use memory-mapped files, RAM drives, or packaged formats like TFRecords / HDF5 / LMDB.
- **Error Handling:** Wrap `cv2.imread()` in explicit try-except blocks to gracefully log corrupt images without terminating the pipeline.

### QUESTION 3

#### What would you check if `cv2.imread()` returns `None` while reading an image?

When `cv2.imread()` returns `None`, OpenCV failed to load the image silently. Check the following root causes:

- **Incorrect File Path:** Verify the absolute/relative path and check for typos, missing file extensions, or case sensitivity.
- **Unsupported Non-ASCII Paths:** In Windows environments, OpenCV's C++ back-end can fail when reading paths containing Unicode/special characters. (Workaround: Use `numpy.fromfile()` + `cv2.imdecode()`).
- **Corrupt File or Invalid Codec:** The file extension might say `.jpg`, but the file structure is broken or saved in an unsupported format/compression.
- **File Permissions:** Ensure the user process has read access permissions for the designated directory/file.
- **Missing OpenCV Codec Support:** Verify if OpenCV was compiled without support for specific image codecs (e.g., LibTIFF, OpenEXR, WebP).

### QUESTION 4

#### Suppose a video is not opening using `cv2.VideoCapture()`. How would you systematically identify the problem?

Systematic debugging steps for `cv2.VideoCapture()` failure:

1. **Check Source Validity:**
  - *File Source:* Verify file existence, read permissions, and absolute path.
  - *Camera/Webcam Source:* Verify index (0, 1, 2) and check if another process is locking the device.
  - *RTSP/Network Stream:* Ping the camera IP, verify stream URL format, and test basic auth credentials.
2. **Verify Codec & Backend Support:** Check if FFmpeg/GStreamer is properly configured. Test video playback using external tools (like VLC) to verify container format (MP4, AVI) and codec (H.264, HEVC).
3. **Force Backend API:** Specify explicit backends in OpenCV, e.g., `cv2.VideoCapture(url, cv2.CAP_FFMPEG)` or `cv2.CAP_DSHOW` on Windows.
4. **Inspect Capture Initialization:** Use `cap.isOpened()` right after initialization to confirm status.

**QUESTION 5**

What is the difference between processing an image and processing a video stream in OpenCV?

| Aspect               | Image Processing                                                   | Video Stream Processing                                       |
|----------------------|--------------------------------------------------------------------|---------------------------------------------------------------|
| Data Structure       | Single static 2D/3D matrix array ( <code>`numpy.ndarray`</code> ). | Continuous sequential sequence of frames (time series).       |
| Latency Constraints  | Offline; latency tolerance is typically high.                      | Real-time constraints (must meet FPS speed target).           |
| Temporal Information | No temporal context available.                                     | Leverages inter-frame tracking, optical flow, motion vectors. |
| Resource Management  | Load and release memory per image file.                            | Requires threaded frame buffer queues, hardware decoding.     |

**QUESTION 6**

If your video-processing application is dropping frames, what could be the reasons, and how would you improve its performance?

**Primary Causes of Frame Drops:**

- **CPU/GPU Bottleneck:** Algorithmic workload exceeds per-frame time budget (e.g., >33.3ms for 30 FPS).
- **Single-Threaded I/O Block:** Frame reading (``cap.read()``) blocks execution while waiting for decoding or network buffers.
- **Memory Allocation Overhead:** Frequent memory re-allocation inside the processing loop.

**Performance Improvement Strategies:**

- **Separate Decoding from Processing:** Use a dedicated background thread for fetching frames into a thread-safe Queue (Producer-Consumer architecture).
- **Downscaling & ROI Processing:** Resize frames before compute-heavy algorithms or restrict execution to a region of interest (ROI).
- **GPU Acceleration:** Utilize CUDA-enabled OpenCV modules (``cv2.cuda``) or OpenVINO / TensorRT execution providers.
- **Frame Skipping:** Process heavy models only on sub-sampled frames (e.g., every 3rd or 5th frame).

### QUESTION 7

**How would you process only every 5th frame of a video? What are the advantages and disadvantages of doing this?**

**Implementation:** Maintain a frame counter variable or use `cap.grab()` (which fast-forwards decoding without decoding full image buffers) 4 times and call `cap.retrieve()` on the 5th frame.

```
frame_id += 1; if frame_id % 5 != 0: continue
```

#### Advantages:

- **5x Processing Speedup:** Cuts computational load drastically, allowing heavy AI models to run on live high-FPS streams.
- **Lower Resource Usage:** Reduces CPU/GPU thermal output and power usage.

#### Disadvantages:

- **Loss of Temporal Continuity:** Fast-moving objects can cause tracking algorithms (like Lucas-Kanade optical flow) to break due to large displacement.
- **Latency / Jitter:** Visual rendering can appear jerky if intermediate output isn't smoothly interpolated.

### QUESTION 8

**Why might an image appear darker after reading it in OpenCV, and how would you correct the problem?**

#### Causes:

1. **Color Space Mismatch (BGR vs. RGB):** Displaying an OpenCV image in Matplotlib without conversion causes channel swapping (Blue and Red flip), distorting color luminance.
2. **Gamma / Color Profile Mismatch:** RAW or 16-bit images displayed without gamma correction, or loading high dynamic range images without scaling.

#### Correction Methods:

- **Color Conversion:** Convert channel ordering before display: `img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)`.
- **Contrast / Brightness Adjustment:** Apply linear transformation `dst = cv2.convertScaleAbs(src, alpha=1.2, beta=30)` or Histogram Equalization / CLAHE (`cv2.createCLAHE()`).

#### QUESTION 9

**If an OpenCV application has high processing latency, which stages of the pipeline would you investigate first?**

Follow a systematic profiling approach using high-resolution timers (`cv2.getTickCount()` or `time.perf_counter()`):

1. **I/O & Frame Acquisition Stage:** Check if `cap.read()` is stalling due to camera exposure latency or disk read speeds.
2. **Image Preprocessing Stage:** Audit costly operations like large-kernel Gaussian Blurring, Median filtering, or high-resolution spatial warping (`cv2.resize`, `cv2.warpPerspective`).
3. **Feature Extraction / Inference Stage:** Inspect deep learning model execution time or CPU-bound feature detectors (e.g., SIFT, Canny, contour hierarchy extraction).
4. **Rendering & Visualization:** Identify display overhead caused by multiple `cv2.imshow()` calls or heavy vector graphics rendering on high-resolution canvases.

#### QUESTION 10

**How would you design a modular OpenCV application in which preprocessing, detection, tracking, and visualization are independent modules?**

Design a software architecture using Object-Oriented Principles and a Pipeline / Producer-Consumer design pattern:

- **Modular Class Abstraction:**

- **FrameFetcher** : Reads raw video frames into a queue.
- **Preprocessor** : Handles resizing, color space conversions, and noise filtering.
- **Detector** : Runs periodic heavy detection (e.g., YOLO/Haar) and outputs bounding boxes.
- **Tracker** : Maintains ID persistence across frames using lightweight trackers (e.g., Kalman Filter / CSRT / SORT).
- **Visualizer** : Composites bounding boxes, text labels, and overlays onto the original frame.

- **Inter-Module Communication:** Pass standardized Data Transfer Objects (DTOs) or dictionaries (containing image state, standard bounding boxes, IDs) through multi-threaded queues.

#### QUESTION 11

**What is the difference between edge detection and corner detection? Give a practical application for each.**

**Differences:**

- **Edge Detection (1D Intensity Changes):** Detects boundaries where intensity changes sharply in one dominant direction.  
*Practical Application:* Document scanner border detection, lane line detection for autonomous driving (Canny Edge Detector).
- **Corner Detection (2D Intensity Changes):** Detects point features where intensity changes significantly in all directions (intersection of edges).  
*Practical Application:* Camera calibration (checkerboard corners using Harris or Shi-Tomasi), image stitching, and visual SLAM.

#### QUESTION 12

**If a feature detector produces thousands of keypoints, how would you reduce the number of irrelevant features?**

To reduce noise and keep key distinctive features:

- **Increase Detection Thresholds:** Raise response/contrast thresholds (e.g., `contrastThreshold` in SIFT or `threshold` in FAST/ORB).
- **Limit Maximum Keypoints:** Set `nfeatures` parameter (e.g., `cv2.ORB_create(nfeatures=500)`).
- **Non-Maximum Suppression (NMS):** Suppress spatial neighbors with lower response scores.
- **ANMS (Adaptive Non-Maximal Suppression):** Select keypoints evenly distributed spatially across the entire image to prevent clustering in high-texture areas.
- **Region Masking:** Apply a mask to ignore irrelevant background/sky regions.

#### QUESTION 13

**How would you compare two images and determine whether they contain significant differences?**

The approach depends on alignment and domain requirements:

1. **Pixel-level Difference (Aligned Images):** Convert images to grayscale, compute absolute difference using `cv2.absdiff(img1, img2)`, apply global thresholding, and compute the ratio of changed non-zero pixels.
2. **Structural Similarity Index (SSIM):** Use SSIM (`skimage.metrics.structural_similarity`) to evaluate perception-based degradation (luminance, contrast, structural noise).
3. **Feature Matching (Un-aligned Images):** Extract ORB/SIFT descriptors, perform feature matching (FLANN/BFMatcher), calculate inliers using RANSAC Homography, and evaluate match quality.

#### QUESTION 14

**Suppose feature matching works for two images taken from the same viewpoint but fails when one image is rotated. How would you improve the system?**

**Root Cause:** The current descriptor/detector (e.g., standard Harris corners or simple Template Matching) lacks rotation invariance.

**Improvements:**

- **Switch to Rotation-Invariant Detectors:** Use **SIFT**, **SURF**, or **ORB**, which compute a canonical orientation (dominant gradient direction) for keypoints before calculating descriptors.
- **Ratio Testing:** Apply Lowe's Ratio Test ( `matches[0].distance < 0.75 * matches[1].distance` ) to remove ambiguous matches.
- **Geometric Verification:** Use `cv2.findHomography()` with RANSAC to filter out false positive matches geometrically.

#### QUESTION 15

**How would you design a system to track important features from one video frame to the next?**

A standard feature-tracking pipeline comprises:

1. **Feature Selection:** Detect strong corners in Frame 1 using Shi-Tomasi Corner Detector ( `cv2.goodFeaturesToTrack()` ).
2. **Sparse Optical Flow Tracking:** Track keypoints into Frame 2 using Lucas-Kanade Optical Flow ( `cv2.calcOpticalFlowPyrLK()` ).
3. **Quality Control & Re-detection:** Filter out keypoints where tracking status fails or forward-backward error exceeds a threshold. When the total tracked count drops below a limit (e.g., < 50 points), re-trigger detection.

#### QUESTION 16

**What are the major challenges in face detection when multiple people are present in a scene? How would you address them?**

**Challenges & Solutions:**

- **Partial Occlusion & Overlap:** Deep learning single-shot detectors (e.g., RetinaFace, YuNet, MTCNN) handle occlusions far better than traditional Haar Cascades.
- **Scale Variation (Small/Far Faces):** Use Feature Pyramid Networks (FPN) inside modern detectors or multi-scale input image pyramids.
- **Pose & Angle Variations:** Train detectors on augmented datasets with extreme roll/pitch/yaw angles.
- **Lighting/Shadow Variations:** Apply histogram equalization (CLAHE) or normalized lighting adjustments during preprocessing.

#### QUESTION 17

**How would you design a simple face-recognition system using OpenCV? Explain the complete pipeline from image acquisition to identification.**

**End-to-End Recognition Pipeline:**

1. **Acquisition:** Capture video frame via `cv2.VideoCapture()`.
2. **Detection & Alignment:** Detect faces using `YuNet` or Haar Cascade. Align faces using eye landmarks (affine warp so eyes are horizontally level).
3. **Preprocessing:** Crop face bounding box, resize to fixed dimensions (e.g., 112x112), and normalize pixel intensities.
4. **Feature Extraction (Embeddings):** Pass crop through a deep feature extractor (e.g., SFace in OpenCV or MobileFaceNet) to obtain a 128D/512D feature vector.
5. **Matching & Identification:** Compute Cosine Distance or Euclidean Distance between candidate vector and database vectors. Assign identity if distance is below predefined threshold.

#### QUESTION 18

**A face-recognition system incorrectly identifies an unknown person as a known person. What changes would you make to reduce false recognition?**

To reduce **False Accept Rate (FAR)**:

- **Tighten Distance Threshold:** Lower the similarity threshold for cosine distance / Euclidean distance matching.
- **Add an "Unknown" Class/Reject Option:** Enforce explicit classification rejection if confidence score fails to clear the decision threshold.
- **Upgrade Feature Extractor:** Replace basic LBPH/Eigenfaces models with modern deep learning embeddings (ArcFace, SFace, Facenet).
- **Liveness Detection:** Implement anti-spoofing checks (blink detection, texture analysis) to eliminate fake presentation attacks.
- **Multi-Frame Verification:** Require consistent identification match across N consecutive frames before accepting identity.

#### QUESTION 19

**How would you improve face-recognition performance when images are captured under different lighting conditions?**

Techniques to achieve illumination invariance:

- **CLAHE (Contrast Limited Adaptive Histogram Equalization):** Equalize local contrast without over-amplifying uniform background noise.
- **Gamma Correction:** Non-linear luminance scaling to correct over-exposed or dark shadow areas.
- **Difference of Gaussians (DoG) Filtering:** Remove high-frequency noise and low-frequency illumination gradients.
- **Color Space Conversion:** Convert to YCrCb or LAB color space and perform equalization solely on the Luminance (Y or L) channel.



#### QUESTION 20

**What preprocessing techniques would you apply to a noisy scanned document before sending it to an OCR system?**

Standard OCR Preprocessing Pipeline:

1. **Grayscale Conversion:** `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` .
2. **Denoising:** Apply Non-Local Means Denoising ( `cv2.fastNlMeansDenoising()` ) or Gaussian Blur to eliminate scanner dust/artifacts.
3. **Adaptive Thresholding:** Use `cv2.adaptiveThreshold()` with Otsu or Gaussian windowing to handle uneven page illumination.
4. **Morphological Operations:** Perform Opening/Closing ( `cv2.morphologyEx()` ) to remove isolated salt-and-pepper noise and bridge broken character strokes.

#### QUESTION 21

**If text in an image is rotated or captured at an angle, how would you prepare the image for OCR?**

Deskewing and Perspective Correction Pipeline:

- **Angle Estimation:**
  - Find minimum bounding area rectangle around text contours ( `cv2.minAreaRect()` ) and extract rotation angle.
  - Alternatively, compute Hough Transform lines ( `cv2.HoughLinesP()` ) and measure dominant orientation.
- **Image Rotation:** Rotate image around center using affine transform matrix ( `cv2.getRotationMatrix2D()` ) and `cv2.warpAffine()` ).
- **Perspective Correction:** If document is at 3D angle, locate 4 document corners, define target rectangle, and apply Homography Warp ( `cv2.getPerspectiveTransform()` + `cv2.warpPerspective()` ).

#### QUESTION 22

**How would you design a real-time system that detects moving objects in a surveillance video?**

Architecture for Motion Detection:

1. **Background Subtraction:** Apply MOG2 or KNN background subtractor ( `cv2.createBackgroundSubtractorMOG2()` ) to isolate foreground masks.
2. **Noise Removal:** Apply morphological opening and Gaussian smoothing to discard small pixel flickers.
3. **Contour Detection:** Find contours on binary foreground mask ( `cv2.findContours()` ).
4. **Bounding Box Filtering:** Filter contours by minimum area threshold to drop noise, and draw bounding boxes around valid moving objects.

#### QUESTION 23

**Suppose a surveillance system generates many false motion detections because of moving trees and changing shadows. How would you reduce these false alarms?**

Strategies to eliminate environmental false positives:

- **Enable Shadow Detection in MOG2:** Set `detectShadows=True` in MOG2 background subtractor and filter out grey shadow pixels.
- **Semantic Object Detection Filter:** Downstream foreground blobs through a lightweight CNN (e.g., MobileNet-SSD/YOLO-nano) to verify if the motion belongs to actual target classes (person, vehicle) rather than trees.
- **Learning Rate Tuning:** Adjust background model learning rate dynamically to integrate swaying foliage into the static background.
- **Masking Invariant ROIs:** Mask out non-critical region areas (like tree canopies or sky) using binary ROI masks.

#### QUESTION 24

**How would you design a people-counting system for a doorway? What problems could occur because of occlusion, people stopping, or changing direction?**

**Design:** Overhead camera angle, define virtual counting line/tripwire. Use object tracker (Centroid Tracker / ByteTrack / DeepSORT) to assign unique IDs and track Y-coordinate trajectories crossing the virtual line.

**Problems & Mitigation Strategies:**

- **Occlusion (Multiple people entering together):** Overhead camera positioning minimizes horizontal occlusion. Depth sensors or head-detection models prevent single bounding box merging.
- **People Stopping / Loitering near door:** Track motion vector history; trigger count event only when trajectory vector fully traverses across state boundaries (IN zone -> OUT zone).
- **Direction Changes:** Maintain state machine tracking directional movement across multi-frame buffer vectors.

### QUESTION 25

Imagine you are asked to develop a real-world Computer Vision application within one week using OpenCV. What application would you choose, and how would you design its complete pipeline? Explain why you selected each technique.

**Chosen Project: Automatic License Plate Recognition (ALPR) System for Parking Access**

#### **Pipeline Architecture & Justification:**

- 1. Image Acquisition:** Capture frame from IP camera feed.  
*Justification:* Inexpensive standard RTSP setup.
- 2. Plate Localization (Color/Edge + Contours):** Convert to Grayscale, apply Sobel filter, threshold, and find rectangular contours with specific aspect ratio (~4:1).  
*Justification:* Lightweight and executes in <5ms on CPU, avoiding deep neural net overhead for initial detection.
- 3. Perspective Correction:** Extract 4 corners of license plate and apply `cv2.getPerspectiveTransform()`.  
*Justification:* Normalizes skewed angles to flat horizontal alignment for better character recognition.
- 4. Character Segmentation & Preprocessing:** Adaptive thresholding + Morphological filtering.  
*Justification:* Isolates individual alphanumeric characters cleanly under varying lighting.
- 5. OCR Recognition:** Pass cropped plate image to Tesseract OCR engine or lightweight ONNX CRNN model.  
*Justification:* Highly accurate pre-trained models readily available, allowing production delivery within a 1-week timeline.